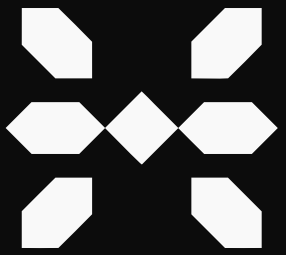
HOOX · AXIS · DOCUMENTATION



AUDITORS · QA · CONTRIBUTORS

REFERENCE MANUAL

Feature atlas, legacy shell, testing, plugin examples

Open charting PWA — own the axes, swap the engine

CONTENTS

01 Feature Atlas

02 Commercial

03 Open Capability Gaps

04 Legacy Shell

05 Testing

06 Plugin Examples

FEATURE ATLAS

COPYRIGHT (C) 2026 HOOX · AXIS · HOOX-SH (JANGO_BLOCKCHAINED)
THIS FILE IS PART OF AXIS.
AXIS IS FREE SOFTWARE: YOU CAN REDISTRIBUTE IT AND/OR MODIFY
IT UNDER THE TERMS OF THE GNU AFFERO GENERAL PUBLIC LICENSE AS PUBLISHED BY
THE FREE SOFTWARE FOUNDATION, EITHER VERSION 3 OF THE LICENSE, OR
(AT YOUR OPTION) ANY LATER VERSION.
AXIS IS DISTRIBUTED IN THE HOPE THAT IT WILL BE USEFUL,
BUT WITHOUT ANY WARRANTY; WITHOUT EVEN THE IMPLIED WARRANTY OF
MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. SEE THE
GNU AFFERO GENERAL PUBLIC LICENSE FOR MORE DETAILS.
YOU SHOULD HAVE RECEIVED A COPY OF THE GNU AFFERO GENERAL PUBLIC LICENSE
ALONG WITH AXIS. IF NOT, SEE HTTPS://WWW.GNU.ORG/LICENSES/.
SPDX-LICENSE-IDENTIFIER: AGPL-3.0-ONLY

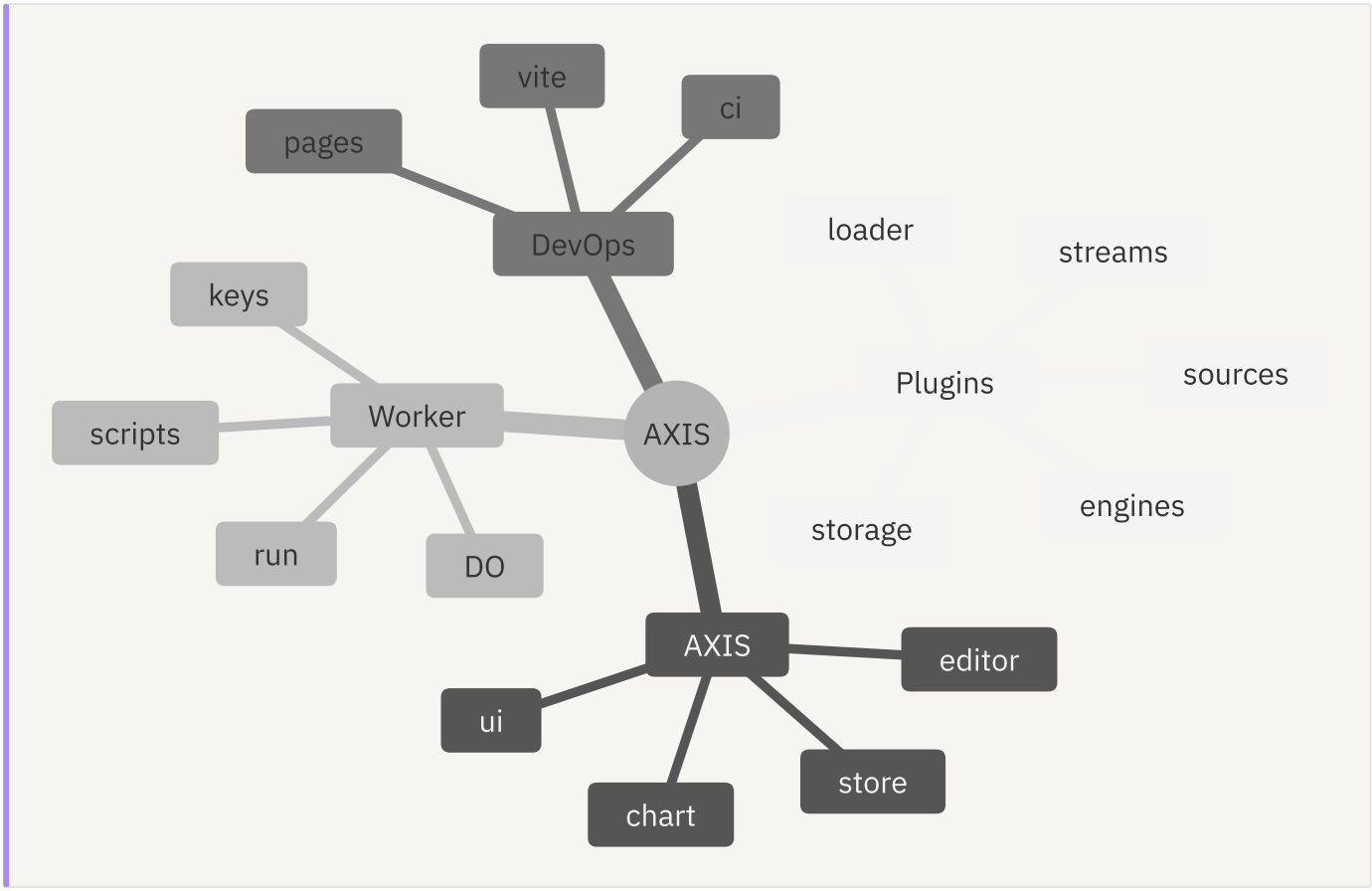
title: "Feature atlas" description: "Map of AXIS product surfaces to repository paths: plugins, AXIS, worker, storage, test, legacy."

FEATURE ATLAS

Abstract

This atlas is a **path index** for AXIS. Use it to jump from a product feature to the code that implements it. Prefer Solid product paths over legacy static shell files.

Conceptual model



Atlas tables

Plugin system

Feature	Primary paths
Contracts	`src/plugins/types.ts`
Registry	`src/plugins/registry.ts`
Bootstrap	`src/plugins/bootstrap.ts`
Active set	`src/plugins/active.ts`
Dynamic install	`src/plugins/loader.ts` (source/stream/engine/dataset/**component**)
Barrel	`src/plugins/index.ts`
Manager UI	`src/ui/PluginManager.tsx` , `plugin-badges.tsx`
Docs (in-tree)	`src/plugins/README.md`

Data plugins

Feature	Primary paths
Sources	`src/sources/catalog.ts`, `upload-store.ts`
Streams	`src/streams/catalog.ts`, `multiplex.ts`
Engines	`src/engines/catalog.ts`
Load symbol (one-shot)	`src/data/load-symbol.ts`, `parse-bars.ts`
Data Source Manager	`src/data/data-source-manager.ts`, `bars-cache.ts`, `bars-gaps.ts`, `ui/DataSourceManagerPanel.tsx`
Script run	`src/indicators/runner.ts`
Scripts panel	`src/indicators/IndicatorPanel.tsx`, `IndicatorCard.tsx`
Built-in script catalog	`src/indicators/builtins/`
Chart themes	`src/theme/presets.ts`, `catalog.ts`, `manager.ts`, `ui/ThemePanel.tsx`

Storage / library

Feature	Primary paths
Catalog	`src/storage/catalog.ts`
Local IDB	`src/storage/local.ts`, `idb.ts`
Cloud	`src/storage/cloud.ts`
Git	`src/storage/git.ts`, `git-github.ts`, `git-gitlab.ts`
Service façade	`src/storage/service.ts`
Library panel	`src/ui/ScriptLibraryPanel.tsx`
v6 starter pack	`examples/script-library-starter.json`
First-party built-in scripts	`src/indicators/builtins/`

UI (UI)

Feature	Primary paths
Entry	`src/index.tsx`, `src/app.tsx`
Chart host	`src/chart/ChartHost.tsx`, `ChartWorkspace.tsx`, `pane-manager.ts`, `pane-badge.ts`, `series-factory.ts`
Price-scale decimals	`src/chart/price-precision.ts`
Plot style parity	`src/results/plot-visuals.ts` → pane-manager
Multi-chart / recipes	`src/chart/layout.ts`, `layout-recipes.ts`, `chart-registry.ts`, `ui/ChartLayoutMenu.tsx`
Bar replay	`src/chart/bar-replay.ts`, `ui/BarReplayControls.tsx`
Compare / volume profile	`src/chart/compare-overlay.ts`, `volume-profile.ts`
Drawings	`src/chart/drawing-layer.ts`, `drawings/*`, `pyne-drawings.ts`, `DrawingToolbar.tsx`
Pine tables panel	`src/chart/pine-tables.ts`, `PyneTableHud.tsx`, `ui/TablesPanel.tsx`, `store.pineTablesLocation`
Extras (ticker · price/time panels)	`src/ui/extras/*` (`PriceTicker.tsx`, `CurrentPriceCard.tsx`, `timebadge.ts`, `trend.ts`), `ui/PricePanel.tsx`, `ui/TimePanel.tsx`
Watchlist rows	`src/ui/Watchlist.tsx`, `ui/watchlist-row.ts`
Panel motion	`src/ui/effects/registry.ts`, `ui/panels/FloatableShell.tsx`, `DockColumn.tsx`
Editor	`src/editor/*` (CM6, diagnostics, ruler, git-sync, inline-debug, symbols)
Editor chrome	`ui/EditorGitBar.tsx`, `ui/EditorProblems.tsx`
Alerts	`src/alerts/*`, `ui/AlertsPanel.tsx`
Command palette	`ui/CommandPalette.tsx`, `command-registry.ts`
Docks	`ui/panels/*` (side-by-side left/right)
Results	`src/results/*`, `ui/ResultsPanel.tsx`, `StrategyReport.tsx`
Workers Manager	`src/ui/WorkersManager.tsx`, `src/workers/*`
Store	`src/store/index.ts`, `types.ts`
Topbar / settings	`ui/Topbar.tsx`, `TopbarField.tsx`, `SettingsDialog.tsx`
Workspace snapshot	`src/storage/workspace-snapshot.ts`
Desktop shell bridge	`src/desktop/*`, `src-tauri/`

Worker

Feature	Primary paths
Router / CORS	`worker/src/index.ts` (`pickOrigin`)
Shared HTTP / proxy / limits	`worker/src/http.ts`, `proxy-router.ts`, `rate-limit.ts`
Client throttle	`src/utils/throttle.ts`
Run (auth + rate limit)	`worker/src/runtime.ts`, `pyodide_runtime.ts`
Auth / keys	`worker/src/auth.ts`, `keys.ts`
Scripts	`worker/src/scripts.ts`, `schemas/scripts.sql`
Git OAuth	`worker/src/git-oauth.ts`
Session DO	`worker/src/durable-objects/session.ts`
Config	`worker/wrangler.toml.example`, `RUNTIME.md`, `README.md`
MCP server	`worker/src/mcp/` (JSON-RPC `/mcp` + `McpBridgeDO`)
MCP PWA host	`src/mcp/` (capabilities, snapshot, WS bridge)
MCP CLI	`packages/cli/src/commands/mcp.ts` (`axis mcp`)

Build / ops

Feature	Primary paths
Package scripts	root `package.json` (`dev`, `build`, `test:*`, `axis:*`, `desktop:*`)
AXIS CLI	`packages/cli/` (`@hoox-sh/axis-cli`)
Static server	`axis_pwa_server.py`
Public assets	`public/plugins`, `public/vendor`, `public/pyodide`
Makefile	`axis` / `axis-*`, `pages-deploy`, `docker-*`
Desktop CI	`.github/workflows/desktop.yml`
CI	`.github/workflows/ci.yml` (coverage + e2e smoke)
Coverage gate	`scripts/check-coverage.mjs` (scoped core ≥95%)
Component lab	`.storybook/`, `src/stories/` (`bun run storybook` :6006)
Harden/perf audit	`docs/devops/harden-perf-audit-2026-08-11.md`

Examples & tests

Feature	Primary paths
Example plugins	`public/plugins/*.js`
Pine v6 library starter	`examples/script-library-starter.json`
First-party built-in Pine	`src/indicators/builtins/`
Unit tests	`tests/**`
Worker tests	`worker/tests/**`
E2E	`e2e/**`, `playwright.config.ts`
Testing guide	`TESTING.md`
Legacy notes	`LEGACY.md`

Namespaces & keys

Key / namespace	Use
`pynescript.axis.plugins.v1`	Installed dynamic plugins
`pynescript.axis.library.v1`	Local library LS fallback
`pynescript.axis.storage`	IDB database name
`pynescript.axis.v1`	App state
`pynescript.axis.v2`	Older app-state key (write-forward)
Contract ns	`pynescript.axis.plugins.v1` (conceptual)

Infrastructure constants

Name	Value
Pages project	`axis`
Pages URL	`https://axis.hoox.sh`
Worker script	`worker-axis`
Worker URL	`https://worker.axis.hoox.sh`
Health service	`worker-axis`
Pyodide version	`0.29.5` (self-hosted path)

See also

- Plugins
- Worker
- Legacy shell

COMMERCIAL

COPYRIGHT (C) 2026 HOOX · AXIS · HOOX-SH (JANGO_BLOCKCHAINED)
THIS FILE IS PART OF AXIS.
AXIS IS FREE SOFTWARE: YOU CAN REDISTRIBUTE IT AND/OR MODIFY
IT UNDER THE TERMS OF THE GNU AFFERO GENERAL PUBLIC LICENSE AS PUBLISHED BY
THE FREE SOFTWARE FOUNDATION, VERSION 3 ONLY.
AXIS IS DISTRIBUTED IN THE HOPE THAT IT WILL BE USEFUL,
BUT WITHOUT ANY WARRANTY; WITHOUT EVEN THE IMPLIED WARRANTY OF
MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. SEE THE
GNU AFFERO GENERAL PUBLIC LICENSE FOR MORE DETAILS.
YOU SHOULD HAVE RECEIVED A COPY OF THE GNU AFFERO GENERAL PUBLIC LICENSE
ALONG WITH AXIS. IF NOT, SEE HTTPS://WWW.GNU.ORG/LICENSES/.
SPDX-LICENSE-IDENTIFIER: AGPL-3.0-ONLY

title: "Commercial & Pro SaaS" description: "AXIS plans: upcoming Commercial licence (closed use) and upcoming Pro SaaS hosting (Starter/Pro/Team)."

COMMERCIAL & PRO SAAS

Abstract

AXIS is **AGPL-3.0-only**. Two paid tracks sit beside the free self-host:

1. **Commercial** (upcoming) — AGPL escape hatch for closed use.
2. **Pro SaaS** (upcoming) — hosted `axis.hoox.sh` so you skip ops.

Track 1 — Commercial (upcoming)

Upcoming · per organisation · lifetime 2.x line.

AGPL requires	Commercial waives
Publish modifications	Keep them private
Offer Corresponding Source to hosted users (§13)	No source offer to your users
Closed / white-label distribution forbidden	Ship OEM inside closed products

Scope: 1 legal entity, unlimited devs / products / domains. No warranty, support, SLA, or hosting quota — legal permission only. Full text: `COMMERCIAL-LICENSE.txt`.

Buy: sales open soon — register interest at sales@hoox.dev.

Track 2 — Pro SaaS (upcoming)

Hosted PWA + Worker (KV keys/usage, D1 script library, Durable Object streams). You bring the browser; we run the edge.

Plan	Price	For	Includes
Starter	SaaS seat — see /membership	Single trader	Hosted PWA, cloud script library, DO live streams, community support
Pro	SaaS seat — see /membership	Active desk	Everything in Starter + multi-device sync, priority evaluate capacity, MCP agent access, email support
Team	SaaS seat — see /membership	Small desk	Everything in Pro + 5 seats, shared workspaces, roles, audit trail

Status: **upcoming** — join via console.hoox.sh waitlist (Notify me on hoox.sh/axis); self-host free under AGPL today (`bun run dev` + Flask `:5002` or offline Pyodide). Team via contact at sales@hoox.dev until self-serve lands.

Which track?

- Embedding AXIS closed-source or hosting it for third parties without AGPL duties → **Commercial**.
- Want hosted charts, sync, and streams without operating Workers → **Pro SaaS** (when live).
- Both → buy both; they are orthogonal (licence ≠ hosting quota).

FAQ

Is self-host free? Yes — AGPL-3.0-only, forever. Paid tracks are convenience (SaaS) or closed-use rights (Commercial).

Does Commercial include SaaS quota? No. Hosting seats/evaluate capacity are Pro SaaS only.

VAT? Merchant-of-record and EU VAT handling will be announced at launch. **Refunds?** 14-day EU digital-goods withdrawal terms will be published at launch.

OPEN CAPABILITY GAPS

COPYRIGHT (C) 2026 HOOX · AXIS · HOOX-SH (JANGO_BLOCKCHAINED)
THIS FILE IS PART OF AXIS.
AXIS IS FREE SOFTWARE: YOU CAN REDISTRIBUTE IT AND/OR MODIFY
IT UNDER THE TERMS OF THE GNU AFFERO GENERAL PUBLIC LICENSE AS PUBLISHED BY
THE FREE SOFTWARE FOUNDATION, EITHER VERSION 3 OF THE LICENSE, OR
(AT YOUR OPTION) ANY LATER VERSION.
AXIS IS DISTRIBUTED IN THE HOPE THAT IT WILL BE USEFUL,
BUT WITHOUT ANY WARRANTY; WITHOUT EVEN THE IMPLIED WARRANTY OF
MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. SEE THE
GNU AFFERO GENERAL PUBLIC LICENSE FOR MORE DETAILS.
YOU SHOULD HAVE RECEIVED A COPY OF THE GNU AFFERO GENERAL PUBLIC LICENSE
ALONG WITH AXIS. IF NOT, SEE HTTPS://WWW.GNU.ORG/LICENSES/.
SPDX-LICENSE-IDENTIFIER: AGPL-3.0-ONLY

title: "Open capability gaps" description: "Research synthesis of high-demand charting, strategy, and script-host capabilities that remain scarce or incomplete in mainstream platforms—framed for AXIS product direction." sidebarTitle: "Capability gaps"

OPEN CAPABILITY GAPS

Abstract

This page records a **product research synthesis**: roughly thirty concrete capabilities that traders, quant authors, and chart power-users still treat as missing or incomplete on mainstream charting and strategy hosts. It is not a shipped-feature checklist for AXIS and not a competitive teardown of any named vendor.

Use it when prioritizing AXIS surfaces (sources, streams, engines, strategy results, debugging, data plane). Items below are durable demand themes from community threads and documented hard limits—not a public vendor roadmap with delivery status.

How to read this list

Term	Meaning here
Still open	Capability is absent, plan-gated to a partial substitute, or still described as unreliable for professional use
Not a ranking poll	No single global leaderboard exists; order is thematic, then by how often demand shows up as trade-blocking
Status changes	Individual venues or UX affordances can ship later; verify against current product notes before treating any row as forever

Highest-pressure themes overall: true tick and order-flow data, portfolio-grade strategy tooling, live multi-bracket and automated execution, higher cloud/screener ceilings, and a real interactive debugger.

Order flow, volume, and tick data

#	Capability	Why it stays open
1	**True tick volume** under volume profiles (including reliable delta)	Profiles over the same range can disagree on buy/sell aggression; many treat minute-aggregated volume as unsuitable for professional volume work
2	**Real volume footprint** with aggressor-tagged prints	Synthetic or approximate footprints are still not trusted next to tick-native tools
3	**Transaction-level order-flow viewing**	Charts that are primarily bar/minute-based cannot show trade-by-trade flow without true tick history

Live trading, brackets, and execution

#	Capability	Why it stays open
4	**Multi take-profit / multi-bracket management on live broker routes**	Multi-exit levels often exist in paper/simulators while live multi-exit brackets remain incomplete or broker-dependent
5	**Direct automated live order placement from strategy scripts**	Strategy engines commonly fill via a broker *emulator*; real exchange orders need external webhooks or bridges
6	**Unified routing** of strategy orders through the same trading panel path as manual/paper orders	Script strategies and discretionary trading panels are often separate stacks

Strategy model: portfolio, hedge, and fill realism

#	Capability	Why it stays open
7	**Native multi-symbol / portfolio strategy backtesting**	One script run is typically bound to one chart dataset; multi-asset P&L is export-and-compare, not a portfolio engine
8	**Simultaneous long + short (hedge) on the same symbol**	Position models are often single-direction at a time
9	**Positions in symbols other than the chart asset**	Cross-asset entries from one strategy remain unsupported on many hosts
10	**Full tick-level historical fill realism**	Fills usually walk chart OHLC with assumed open→high→low→close (or open→low→high→close) paths; higher-resolution path reconstruction is partial and often tier-gated
11	**Large trade histories without silent trim**	Non–deep backtests often cap order history (older trades dropped from testers); deep modes raise the ceiling but remain product-tier features

Script runtime, quotas, and tooling

#	Capability	Why it stays open
12	**Higher unique multi-series / remote-series request ceilings**	Cloud hosts hard-cap distinct series fetches per script (plan tiers only raise the number)
13	**Higher total script wall-time budgets**	Entire-run timeouts remain fixed per plan class
14	**Higher per-bar loop budgets**	Tight loop-per-bar limits reject heavy iterative logic
15	**Higher data, memory, and compiled-size caps**	Large libraries and data-heavy studies hit opaque resource walls
16	**Large multi-symbol scanning without burning unique-request quotas**	Dynamic loops still count each distinct symbol/timeframe against the unique ceiling
17	**Runtime profilers** (wall time, memory, compiled size) before failure	Authors discover limits by rejection or runtime error, not by measurement tools
18	**Interactive step-through debugger with breakpoints**	Official debugging remains logs, plots, drawings, and chart colors—not a classic debugger

Script-driven screener constraints

#	Capability	Why it stays open
19	**Multiple custom indicators per screener pass**	Many screeners allow only one user script per screen
20	**Indicator-on-indicator** composition inside the screener	Nested studies are unsupported
21	**Custom timeframes** in scripted screens	Non-standard bars often unavailable
22	**Higher remote-series call counts** inside screener scripts	Typical caps are very small (single-digit)
23	**Lookback beyond a short trailing window** (e.g. hundreds of bars only)	Screen math is truncated to recent history
24	**Full input-type coverage** for screener-hosted scripts	Several input kinds are unsupported in the screener surface

Chart UX and market coverage

#	Capability	Why it stays open
25	**Stable, discoverable watchlist gestures** (e.g. context-menu add)	Removal or relocation of common chart chrome produces sustained restore demand
26	**Timely exchange / venue chart coverage** when a market is popular	Coverage gaps surface as high-engagement “still missing” demand until a data feed lands
27	**Sector / industry / theme heatmaps** and similar multi-name overview boards	Recurring UX requests outside pure charting

Open script host vs general-purpose stacks

#	Capability	Why it stays open
28	**External data sources, HTTP, and host APIs from inside scripts**	Cloud sandboxes intentionally block outbound and arbitrary I/O
29	**Databases, file I/O, and machine-learning libraries inside the script runtime**	Language surface is chart-bound, not a general quant stack
30	**True custom UI and host programming** beyond chart plots and fixed panels	Outgrow path is usually a multi-asset engine with live broker APIs outside the chart host

---

Themes for AXIS

These gaps map cleanly onto AXIS axes without implying any third-party product parity:

Demand theme	AXIS lever
Tick / order-flow fidelity	Pluggable **sources** and **streams** with higher-resolution bars or trades when data exists
Portfolio multi-symbol strategies	Multi-source strategy evaluation and results beyond single-chart P&L
Live multi-exit and automation	Optional execution adapters / webhooks (explicit, user-owned—not silent live routing)
Runtime ceilings and profilers	Local and Worker engines with measurable limits you control
Interactive debugger	Step and pin tooling against the active engine (see UI debugging surfaces)
Screener scale	Offline or self-hosted scan loops unbound by a single vendor’s cloud quota
External data / ML / DB	Host bridges and plugins—not inventing closed chart-host APIs

AXIS remains a composition host: sources load history, streams advance the present, engines evaluate scripts, storage holds libraries. Capability work should preserve that separation.

Related docs

- Architecture overview — composition model
- Strategy and results (end user) — how strategy output is surfaced today
- Debugging (UI) — current debug surfaces
- Feature atlas — path index for implementation work

Research notes

- Synthesis date context: community and documentation signals through early–mid 2026.
- “Still open” means encoded as present limits or still treated as incomplete in high-engagement demand—not items already reversed by later product notes.
- Engagement is qualitative across forums and social posts; it is not a weighted global survey.
- Plan-gated historical depth and deep-backtest order ceilings partially address older “more bars / more trades” asks; those are de-emphasized relative to portfolio, execution, order-flow, and tooling gaps.

LEGACY SHELL

COPYRIGHT (C) 2026 HOOX · AXIS · HOOX-SH (JANGO_BLOCKCHAINED)
THIS FILE IS PART OF AXIS.
AXIS IS FREE SOFTWARE: YOU CAN REDISTRIBUTE IT AND/OR MODIFY
IT UNDER THE TERMS OF THE GNU AFFERO GENERAL PUBLIC LICENSE AS PUBLISHED BY
THE FREE SOFTWARE FOUNDATION, EITHER VERSION 3 OF THE LICENSE, OR
(AT YOUR OPTION) ANY LATER VERSION.
AXIS IS DISTRIBUTED IN THE HOPE THAT IT WILL BE USEFUL,
BUT WITHOUT ANY WARRANTY; WITHOUT EVEN THE IMPLIED WARRANTY OF
MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. SEE THE
GNU AFFERO GENERAL PUBLIC LICENSE FOR MORE DETAILS.
YOU SHOULD HAVE RECEIVED A COPY OF THE GNU AFFERO GENERAL PUBLIC LICENSE
ALONG WITH AXIS. IF NOT, SEE HTTPS://WWW.GNU.ORG/LICENSES/.
SPDX-LICENSE-IDENTIFIER: AGPL-3.0-ONLY

title: "Legacy shell" description: "The pre-Solid static shell was removed in 2.21.0 — what was deleted and what replaced it."

LEGACY SHELL (REMOVED)

Abstract

AXIS's product path is **Solid + Vite**. The pre-Solid static shell was **deleted in 2.21.0** (~3,900 lines). Before deletion the import graph was traced from `src/index.tsx`: no legacy file was reachable and none shipped in `dist/`.

Source of truth: `LEGACY.md`.

Conceptual model



What was removed

Deleted	Role
`src/main.js`	Old bootstrap (chart.js, legacy-topbar.js, …)
`src/registry.js`, `src/registry.d.ts`	Pre-Solid plugin registry
`src/registry-bootstrap.js`, `src/plugin-types.d.ts`	Legacy builtin registration + ambient typings
`src/state.js`, `src/state-hash.js`	Pre-Solid store + URL hash hydration
`src/{sources,streams,engines}/index.js`	Legacy builtin plugin barrels
`src/ui/{manager,legacy-topbar,legacy-watchlist,…}.js`	Legacy DOM UI
`pyne-editor.js`	Pre-Solid CM6 wiring
`style.css`, `storage.js`	TV-blue tokens / unused
`server.ts`	Bun static server for old tree
Root `sw.js`	Duplicate of `public/sw.js`
`tests/{registry,state,server}.test.ts`	Tests of deleted files only

There is no `frontend/` directory and no `make run-frontend` target. The product path is `bun run dev`. A static demo of `dist/` is `python axis_pwa_server.py` (default port 8081) after `bun run build`.

What is current

Path	Role
`src/index.tsx`, `app.tsx`	Solid app
`src/chart/ChartHost.tsx`	LWC panes
`src/plugins/registry.ts`	Plugin registry (singleton `registry`)
`src/{sources,streams,engines}/catalog.ts`	Builtin catalogs
`src/ui/*`	Topbar, Settings, Results, Manager, …
`public/` + `bun run build`	Production PWA assets (`public/sw.js` is the only service worker)
`axis_pwa_server.py`	Serve **dist** for demos

Migration residue

Residue	Handling
`pynescript.axis.plugins.v1`	Plugin install list
`pynescript.axis.library.v1`	Library localStorage fallback
`pynescript.axis.editor.doc`	Editor document backup
CF Pages project `axis`	Pages id; canonical `https://axis.hoox.sh`
CF Worker `worker-axis`	Script name; URL `https://worker.axis.hoox.sh`
Some docs/Makefile strings	Current brand is AXIS

App state namespace: `pynescript.axis.v1`.

Invariants

1. **One plugin registry**: `src/plugins/registry.ts`, singleton `registry`. There is no longer a second one.
2. **One service worker**: `public/sw.js`, served at `/sw.js` and registered by `src/pwa/register-sw.ts`.
3. **One state store**: `src/store` (Solid). The pre-Solid `state.js` is gone.
4. **Pages deploy** uses Vite `dist/`, not the repo root static tree.

Failure modes

Confusion	Clarification
“UI doesn’t match docs”	You may be serving a stale build or cache; `bun run build` then reload
Plugin manager missing	A hard reload clears the old shell from cache
Service worker looks stale	Only `public/sw.js` is served; unregister any older root `sw.js`

See also

- Feature atlas
- Build and serve
- AXIS hub

TESTING

COPYRIGHT (C) 2026 HOOX · AXIS · HOOX-SH (JANGO_BLOCKCHAINED)
THIS FILE IS PART OF AXIS.
AXIS IS FREE SOFTWARE: YOU CAN REDISTRIBUTE IT AND/OR MODIFY
IT UNDER THE TERMS OF THE GNU AFFERO GENERAL PUBLIC LICENSE AS PUBLISHED BY
THE FREE SOFTWARE FOUNDATION, EITHER VERSION 3 OF THE LICENSE, OR
(AT YOUR OPTION) ANY LATER VERSION.
AXIS IS DISTRIBUTED IN THE HOPE THAT IT WILL BE USEFUL,
BUT WITHOUT ANY WARRANTY; WITHOUT EVEN THE IMPLIED WARRANTY OF
MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. SEE THE
GNU AFFERO GENERAL PUBLIC LICENSE FOR MORE DETAILS.
YOU SHOULD HAVE RECEIVED A COPY OF THE GNU AFFERO GENERAL PUBLIC LICENSE
ALONG WITH AXIS. IF NOT, SEE HTTPS://WWW.GNU.ORG/LICENSES/.
SPDX-LICENSE-IDENTIFIER: AGPL-3.0-ONLY

title: "Testing reference" description: "AXIS unit, worker, security, coverage gate, and Playwright e2e conventions with paths and commands."

TESTING REFERENCE

Abstract

AXIS tests are **Bun-first** under the repo root, with Playwright for browser smoke and a scoped coverage ratchet. Canonical human guide: `TESTING.md`. This page is the reference map for agents and contributors.

Conceptual model



Commands

```
bun run test                 # unit + worker tests
bun run test:unit            # tests only
bun run test:coverage        # lcov + text → coverage/
bun run test:coverage:gate   # coverage + 95% scoped gate
bun run test:security        # tests/security/
bun run test:e2e:smoke       # Playwright @smoke
bun run test:e2e:critical    # @smoke|@critical
bun run test:e2e             # all
bun run test:all             # gate + security
```

Layout

```
tests/
  setup.ts
  fixtures/
  helpers/
  *.test.ts
  integration/
  security/
scripts/check-coverage.mjs
e2e/
  smoke.spec.ts
playwright.config.ts
worker/tests/
  auth.test.ts
  keys-*.test.ts
  scripts*.test.ts
  …
```

Coverage policy

Item	Policy
Gate	**95%** lines on scoped core
Ratchet history	70 → 80 → 83 → 87 → 90 → **95**
Include	plugins, storage (minus idb), store, results, sources, streams, data, chart pure helpers + series-factory/manager-access, worker auth/keys/runtime/scripts
Exclude from gate	`drawing-layer.ts`, pane-manager (unit elsewhere), runner chart apply, pyodide boot, legacy JS, Solid UI `.tsx`
Full report	`bun test --coverage` unscoped

Conventions

1. `import './setup'` first when tests touch store/plugins/storage.
2. **No real network** — mock `fetch` / `WebSocket`.
3. Table-driven tests for catalog built-ins.
4. Plugin fixtures in `tests/fixtures/plugins/`.
5. Prefer calling handlers with fake params over full browser for unit scope.

Playwright

Config (`playwright.config.ts`):

- `testDir: ./e2e`
- Chromium project
- `webServer`: `bun run build && bun run preview -- --host 127.0.0.1 --port 4173`
- `baseURL`: `http://127.0.0.1:4173` (override `AXIS_E2E_BASE`)
- Timeout 60s; CI retries 1

Smoke mocks `/run` and Binance. Use `data-testid` attributes (`axis-btn-load`, `axis-manager`, ...).

Security suite

`bun run test:security` covers:

- Plugin URL scheme rejection
- Storage-via-URL rejection
- Poisoned localStorage
- Worker partition isolation
- If-Match 409
- Admin keys

Adding a unit test

1. Create `tests/<area>.test.ts`.

2. Import setup if needed.
3. Use fixtures under `tests/fixtures/`.
4. Run `bun run test:unit` and `bun run test:coverage:gate`.

CI linkage

See CI and testing for workflow job names and nightly schedule.

Failure modes

Failure	Mitigation
Coverage drop	Add tests in scoped files or deliberately exclude only with policy change
E2E flake	Mock network; single worker; stable testids
IDB in unit	Local storage falls back to memory maps

See also

- CI and testing
- Plugin contracts
- Worker auth

PLUGIN EXAMPLES

COPYRIGHT (C) 2026 HOOX · AXIS · HOOX-SH (JANGO_BLOCKCHAINED)
THIS FILE IS PART OF AXIS.
AXIS IS FREE SOFTWARE: YOU CAN REDISTRIBUTE IT AND/OR MODIFY
IT UNDER THE TERMS OF THE GNU AFFERO GENERAL PUBLIC LICENSE AS PUBLISHED BY
THE FREE SOFTWARE FOUNDATION, EITHER VERSION 3 OF THE LICENSE, OR
(AT YOUR OPTION) ANY LATER VERSION.
AXIS IS DISTRIBUTED IN THE HOPE THAT IT WILL BE USEFUL,
BUT WITHOUT ANY WARRANTY; WITHOUT EVEN THE IMPLIED WARRANTY OF
MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. SEE THE
GNU AFFERO GENERAL PUBLIC LICENSE FOR MORE DETAILS.
YOU SHOULD HAVE RECEIVED A COPY OF THE GNU AFFERO GENERAL PUBLIC LICENSE
ALONG WITH AXIS. IF NOT, SEE HTTPS://WWW.GNU.ORG/LICENSES/.
SPDX-LICENSE-IDENTIFIER: AGPL-3.0-ONLY

title: "Plugin examples" description: "Worked AXIS example plugins: CoinGecko source, Tiny Pine engine, Cloudflare® DO stream — load paths and contracts."

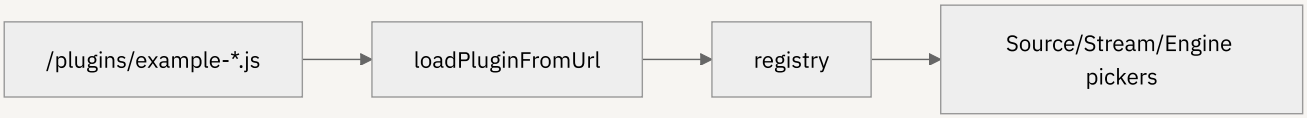
PLUGIN EXAMPLES

Abstract

AXIS ships three **loadable example plugins** as ES modules under `public/plugins/` (also mirrored for reference under `src/plugins/example-*.js`). They are not built-ins; install via Manager → Plugins → **Load from URL**.

Full narrative: `src/plugins/README.md`.

Conceptual model



Loading

1. Serve the PWA (Vite dev, preview, or `axis_pwa_server.py` on `dist`).
2. Open **Manager** → **Plugins**.
3. Paste a same-origin URL, for example:

```
http://127.0.0.1:8081/plugins/example-coingecko-source.js
http://127.0.0.1:3000/plugins/example-tiny-pyne-engine.js
http://127.0.0.1:8081/plugins/example-cf-do-stream.js
```

4. **Load** — registry updates; pickers show the new plugin.
5. Persistence: `localStorage` `pynescript.axis.plugins.v1` restores on next visit.

You may host modules on any origin that allows **module** CORS (same-origin is simplest).

Example catalog

File	id	kind	Purpose
`example-coingecko-source.js`	`coingecko`	source	Public CoinGecko market_chart → synthetic OHLC
`example-tiny-pyne-engine.js`	`tiny-pyne`	engine	In-browser JS DSL: sma/ema/rsi/plot/strategy
`example-cf-do-stream.js`	`cf-do`	stream	WS to Worker `/api/stream` SessionDO
External: `pyne-agent-worker` `/plugin/axis-pine-agent.js`	`pyne-agent`	component	NL → PYNE Agent chat (Cloudflare® Workers AI™); docs

Paths:

- Served: `public/plugins/`
- Source copies: `src/plugins/example-*.js`

CoinGecko source

Contract: `kind: 'source'`, `fetchHistorical`.

Config: `baseUrl` (default CoinGecko v3), `vsCurrency` (default `usd`).

Behavior:

- Maps common symbols (`BTC` → `bitcoin`, …).
- Requests `market_chart` for a day window derived from interval × bar budget.
- Synthesizes OHLC from consecutive price points (API is not full candles).

Limits: public rate limits (~10–30 calls/min); CORS depends on CoinGecko policy.

Use when: demo custom source without a key.

Tiny Pine engine

Contract: `kind: 'engine'`, `isReady`, `run`.

Behavior:

- Tokenizes a tiny expression language.
- Built-ins: `close`, `open`, `high`, `low`, `volume`, `sma`, `ema`, `rsi`, `plot`, `strategy.entry` / `strategy.close`.
- Returns `RunResult` with plots/events — **not** full PYNE fidelity.

Use when: offline demos without loading ~14MB Pyodide or hitting Flask.

Cloudflare® DO stream

Contract: `kind: 'stream'`, `start` → stop.

Config: `endpoint` — Worker base URL (`https://…` or `http://127.0.0.1:8787`). Empty → error.

URL built:

```
{endpoint as wss}/api/stream?session={symbol}@{interval}&symbol=…&interval=…
```

Requires: Worker with `SESSIONS` Durable Object bound and deployed (durable objects).

Use when: many tabs share one upstream Binance kline subscription.

Minimal authoring templates

Source

```
  id: 'my-source',
  name: 'My Source',
  kind: 'source',
  description: 'Shown in Manager',
  configSchema: {
    baseUrl: { type: 'string', default: 'https://example.com', label: 'Base URL' },
  },
  async fetchHistorical({ symbol, interval, config }) {
    const res = await fetch(`${config.baseUrl}/bars?symbol=${symbol}&interval=${interval}`);
    return res.json(); // Bar[]
  },
};
```

Stream

```
  id: 'my-stream',
  name: 'My Stream',
  kind: 'stream',
  start({ symbol, interval, onBar, onError, onStatus }) {
    const ws = new WebSocket('wss://example.com/stream');
    ws.onopen = () => onStatus({ state: 'open' });
    ws.onmessage = (ev) => {
      const b = JSON.parse(ev.data);
      onBar({ time: b.t, open: b.o, high: b.h, low: b.l, close: b.c, volume: b.v });
    };
    ws.onerror = () => onError(new Error('ws error'));
    return () => ws.close();
  },
};
```

Engine

```
  id: 'my-engine',
  name: 'My Engine',
  kind: 'engine',
  async isReady() { return true; },
  async run({ script, bars }) {
    return { status: 'success', plots: bars.map((b) => b.close), events: [], series: {}, meta: {} };
  },
};
```

Storage plugins **cannot** be loaded from URL yet.

Invariants & edge cases

1. Treat plugin URLs as **executable code**.
2. `normalizePluginUrl` rewrites `/src/plugins/` → `/plugins/` for production.
3. Dangerous schemes rejected (`javascript:` , `vbscript:` , `data:text/html`).
4. After load, window may emit product-specific events (see README) for UI extensions.

Failure modes

Issue	Fix
Failed to restore URL	404 after deploy path change
CORS on CoinGecko	Offline mock or proxy
DO stream errors	Bind SessionDO; set endpoint
Tiny engine parse errors	Unsupported DSL constructs

See also

- Dynamic loader
- Contracts
- Worker durable objects